量化入门指南

以「A 股周内效应因子」为例

ShenzhenLime

量化研究

April 6, 2026

ShenzhenLime

目录

1. 第一部分：量化因子策略完整流程
2. 第二部分：代码实际演示
3. 总结与延伸

ShenzhenLime

量化因子策略：总体流程

① 数据获取 → ② 数据预处理 → ③ 因子构建 → ④ 因子评估 → ⑤ 组合回测 → ⑥ 结果分析

核心思想

找到一个能**预测未来收益**的截面变量（因子），利用因子排名构建**多空组合**，并通过严格的统计检验衡量其有效性。

Shenzhen Lime

步骤一：数据获取

需要哪些数据?

- **股票池**：选取目标指数（如中证1000）的成分股
- **交易日历**：确定有效交易日序列
- **个股日线行情**：每日开高低收、涨跌幅
- **财务/估值数据**：市值、市净率（PB）等
- **指数日线行情**：用作基准与定价因子计算

常用数据来源

- Tushare（国内免费/付费（推荐））
- Wind（专业终端）
- Bigquant（量化平台）

注意事项

使用 Jolib 并行下载；需关注 api 接口的**频率限制**；建议以**增量方式**下载；存储方式推荐：数据库（如 DuckDB）以加速查询。

Shenzhen Time

步骤二：数据预处理

时间对齐

- 将原始的日频数据转为特定时段的**(周频、月频)**
- 以周期内最后个交易日表示该时段收益率
- 剔除上市时间不足、停牌等异常个股

收益率计算

$$r_{i,t} = \frac{P_t - P_{t-1}}{P_{t-1}}$$

日度 → 月度：对当月所有日收益率求和

定价因子构建

因子	含义
MKT	市场超额收益（指数日涨跌幅）
SMB	小市值组合 − 大市值组合
HML	高 PB 组合 − 低 PB 组合
UMD	高动量组合 − 低动量组合

分组规则

后三个因子按上月底截面数据三等分（前 1/3、中 1/3、后 1/3），取两端组差值构造因子收益。

Shenzhen Time

步骤三：因子构建——Carhart 四因子模型

Carhart 四因子定价模型：

$$R_{i,t} = \alpha_i + \beta_{i,\mathrm{MKT}}\mathrm{MKT}_t + \beta_{i,\mathrm{SMB}}\mathrm{SMB}_t + \beta_{i,\mathrm{HML}}\mathrm{HML}_t + \beta_{i,\mathrm{UMD}}\mathrm{UMD}_t + \epsilon_{i,t}$$

核心步骤

1. 对每只股票，**分别**用**全历史**数据对周一收益率、周内收益率跑 OLS 回归
2. 估计出两组 $\hat{\beta}$（周末组、周内组）
3. 计算残差：$\hat{\epsilon}_{i,t} = R_{i,t} - \hat{R}_{i,t}$
4. 按月聚合：$\epsilon^{(m)}_{i,\mathrm{wkend}}$，$\epsilon^{(m)}_{i,\mathrm{wkday}}$
5. 最终因子：

$$\boxed{F_{i,m} = \bar{\epsilon}^{(m)}_{i,\mathrm{wkend}} - \bar{\epsilon}^{(m)}_{i,\mathrm{wkday}}}$$

经济含义

$F_{i,m} > 0$：该股周一的**特质波动率**高于周内，说明市场消化了更多**高质量信息**，预示**未来收益更高**。

工程细节

- 至少 50 个交易日才能进行回归
- beta 系数缓存，避免重复计算
- Joblib 多线程并行加速

shenzhenlime

步骤四：因子有效性评估

IC / IR 分析

每期对截面（所有公司）做 Spearman 相关，得到 IC 时间序列。通常用 IC 的均值表示一个因子长期的 IC 表现。

$$\mathrm{IC}_m = \mathrm{Corr}_{\mathrm{Spearman}}(\mathrm{Factor}_{i,m},\ r_{i,m+1})$$

$$\mathrm{IR} = \frac{\overline{\mathrm{IC}}}{\sigma(\mathrm{IC})}$$

- $|\overline{\mathrm{IC}}| > 0.03$ 具有参考价值
- $\mathrm{IR} > 0.5$ 认为信号稳定

分层回测

1. 模拟实际月度调仓（做多高因子值股票，做空低因子值股票）
2. 每月月底，按因子值将股票**等分 10 组**
3. 统计下一个月**各组平均收益**
4. 验证：“从组 1 到组 10 是否单调递增？”

多空组合（L-S）

$$r_{L-S,m} = r_{组\ 10,m} - r_{组\ 1,m}$$

$$\mathrm{SR}_{\mathrm{annual}} = \frac{\bar{r}_{L-S}}{\sigma_{r_{L-S}}} \times \sqrt{12}$$

Shenzhen Line

步骤五：投资组合构建与回测

月度调仓流程

月底：取当月所有周的因子值均值

↓

截面排名，等分 10 组（分位数切分）

↓

多头 = 第 10 组（等权），空头 = 第 1 组（等权）

↓

持仓 1 个月，统计下月收益

↓

下月底再平衡，循环迭代

关键绩效指标

- **IC 均值**：因子预测能力
- **IR**：因子稳定性
- **夏普比率（年化）**：风险调整收益
- **累计超额收益**：L-S 组合累积曲线
- **分层单调性**：组收益是否递增

与基准比较

将多空组合累计收益与**中证 1000 指数**同期表现对比，判断策略是否创造超额价值（Alpha）。

Shen Zhenzhe

步骤六：结果分析与策略诊断

输出内容

文件	内容
`eval_summary`	IC 均值/IR/夏普率汇总
`ic_series`	逐月 IC 时间序列
`group_ret`	十组逐月收益明细
`group_stats`	各组均值/标准差
`ls_ret`	多空组合逐月收益

重要提醒

回测结果不代表实盘表现！**交易成本**、**滑点**、**涨跌停限制**均未纳入。

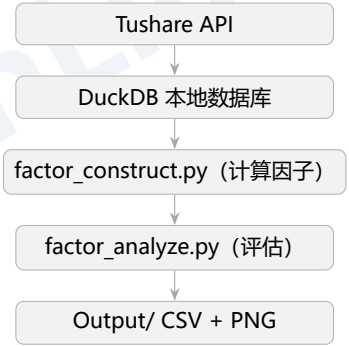
ShenzhenLime

项目结构一览

文件结构

- `main.py` ← 主流程入口、数据下载
- `factor_construct.py` ← 因子计算
- `factor_analyze.py` ← 因子评估
- `db_utils.py` ← 数据库工具
- `Data/` ← 数据存储（DuckDB 未上传至 GitHub）
- `Output/` ← 结果输出

数据流向



ShenZhenLime

代码演示 ①：数据获取——增量下载 + DuckDB

工程思路：先查库中已有哪些日期，只下载缺失部分，避免重复请求。

```python
# 1. 查询 DuckDB 已有的交易日，得到 already_downloaded
existing = db_utils.read_sql("SELECT DISTINCT trade_date FROM stock_bar")
already_downloaded = set(existing['trade_date'].astype(str))

# 2. 与完整交易日历对比，过滤出待下载日期
dates_to_download = [d for d in all_dates if d not in already_downloaded]

# 3. Joblib 多线程并行下载（7 并发，每请求间隔 0.15s 防限流）
results = Parallel(n_jobs=7, prefer="threads")(
    delayed(fetch_bar_by_single_date)(date, TOKEN)
    for date in tqdm(dates_to_download)
)

# 4. 追加写入 DuckDB（append 模式，历史数据不重写）
new_data = pd.concat([df for _, df in results if df is not None])
db_utils.write_to_db(new_data, 'stock_bar', if_exists='append')
```

代码演示 ②：定价因子——以 SMB 为例（HML / UMD 同理）

工程思路：将**上月底**市值 prev_mv 对齐到当月每日，截面排序切三等分，取两端差值；HML/UMD 替换排序字段即可。

```python
# prev_mv = 上月底市值，通过 next_month 提前 merge 到当日
def calc_daily_factors(trade_date, day_data):
    mv = day_data.dropna(subset=['prev_mv']).sort_values('prev_mv')
    n = len(mv) // 3                          # 三等分截断点
    SMB = mv.iloc[:n]['pct_chg'].mean() \     # 后1/3 小市值
        - mv.iloc[-n:]['pct_chg'].mean()      # 前1/3 大市值
    return {'trade_date': trade_date, 'SMB': SMB, ...}

# 按交易日并行，结果写入 DuckDB
results = Parallel(n_jobs=-1, prefer='threads')(
    delayed(calc_daily_factors)(d, g)
    for d, g in df.groupby('trade_date'))
db_utils.write_to_db(pd.DataFrame(results), 'pricing_factors')
```

代码演示 ③：因子构建工程——Beta 缓存 + 并行调度

工程思路：一次性读全量数据进内存；已算过的股票复用缓存 beta，不重新回归。

```python
# 1. 一次性读全量数据（避免多线程并发访问数据库）
all_stock_data = db_utils.query_stock_bar(...)
all_stock_data = all_stock_data.merge(df_pricing_merge, on='date')

# 2. 读取已缓存的 beta（上次运行结果）
try:
    existing_betas = db_utils.read_sql("SELECT * FROM stock_betas")
    existing_beta_dict = existing_betas.set_index('ts_code').to_dict('index')
except:
    existing_beta_dict = {}   # 首次运行，全部重算

# 3. 仅对尚无缓存的股票计算因子
need_calc = [c for c in stock_codes if c not in existing_stocks]
results = Parallel(n_jobs=-1, prefer='threads')(
    delayed(calc_stock_factor)(
        code, stock_groups[code],
        existing_beta_dict.get(code)  # 有缓存则传入，函数内直接跳过回归
    ) for code in tqdm(need_calc)
)
```

代码演示 ④：单股 OLS 回归 + 月度聚合

工程思路：对周一 / 周内分别跑全历史 OLS，月内残差均值之差即为因子值。

```python
def calc_stock_factor(ts_code, stock_df, existing_beta=None):
    if existing_beta:           # 有缓存：直接取 beta，跳过回归
        beta_mon, beta_wkd = load_beta(existing_beta)
    else:                       # 无缓存：用全历史 OLS 估计
        monday_df  = stock_df[stock_df['weekday'] == 0]
        weekday_df = stock_df[stock_df['weekday'] != 0]
        X_mon = np.column_stack([np.ones(len(monday_df)),
                                 monday_df[['MKT','SMB','HML','UMD']]])
        X_wkd = np.column_stack([np.ones(len(weekday_df)),
                                 weekday_df[['MKT','SMB','HML','UMD']]])
        beta_mon = np.linalg.lstsq(X_mon, monday_df['pct_chg'], rcond=None)[0]
        beta_wkd = np.linalg.lstsq(X_wkd, weekday_df['pct_chg'], rcond=None)[0]

    # 残差 → 按月聚合 → 因子 = E[周一残差] - E[周内残差]
    monday_df['resid']  = monday_df['pct_chg']  - X_mon @ beta_mon
    weekday_df['resid'] = weekday_df['pct_chg'] - X_wkd @ beta_wkd
    factor_m = monday_df.groupby('year_month')['resid'].mean() \
             - weekday_df.groupby('year_month')['resid'].mean()
```

代码演示 ⑤：因子评估——分组回测与 IC 计算

工程思路：月底截面分 10 组，统计下月各组收益；同时计算截面 Rank IC。

```python
# 月底截面按因子值等分 10 组
df['group'] = df.groupby('trade_date')['factor'].transform(
    lambda x: pd.qcut(x, 10, labels=range(1,11), duplicates='drop')
)

# 多空收益：第10组（强） - 第1组（弱）
long_ret  = grp_ret[grp_ret['group']==10]['group_ret']
short_ret = grp_ret[grp_ret['group']==1 ]['group_ret']
ls_ret = long_ret.values - short_ret.values

# Rank IC（Spearman 相关）：衡量因子截面预测能力
ic_series = df.groupby('trade_date').apply(
    lambda x: x['factor'].corr(x['ret_next'], method='spearman')
)

# 汇总指标
IR         = ic_series.mean() / ic_series.std()
SR_annual = (ls_ret.mean() / ls_ret.std()) * np.sqrt(12)
```

代码演示 ⑥：一键运行主流程

```python
def main():
    index_code = '000852.SH'  # 中证1000
    use_db = True             # 使用 DuckDB 存储
    get_ins(index_code)                      # 1. 获取成分股
    get_trade('20160101', today)             # 2. 获取交易日历
    get_stock_data_by_date(use_db=True)      # 3. 增量下载股票日线（多线程）
    get_index_data(index_code)               # 4. 获取指数日线（基准）
    get_daily_basic(use_db=True)             # 5. 下载每日财务指标（PB、市值）
    compute_pricing_factors(...)             # 6. 计算 SMB / HML / UMD
    compute_week_effect()                    # 7. 计算周末效应因子（OLS残差）
    evaluate_factor(                         # 8. 因子评估（月度调仓，分10组）
        factor_table='week_factor',
        index_code='000852.SH', n_groups=10)
```

总结：量化因子策略的核心要点

本项目涉及的核心知识点

1. **资产定价模型**：Carhart 四因子框架
2. **OLS 残差**：特质收益率提取
3. **截面分析**：IC/IR、分层回测
4. **多空组合**：夏普率评估
5. **工程实现**：并发、增量、数据库

进一步学习方向

- **组合优化**：均值方差、最大夏普
- **机器学习**：线性回归、决策树
- **高频因子**：分钟级数据、订单流

量化研究 = 金融逻辑 + 统计方法 + 工程实现

Shenzhen time

参考资料

- **Carhart (1997)**： “On persistence in mutual fund performance”，Journal of Finance
- **开源证券**：APM 因子模型的进阶版——市场微观结构研究系列（5）
- **中信建投**：特质波动率纯因子在 A 股的实证研究
- **Tushare** 数据接口：`https://tushare.pro`
- **DuckDB** 列式数据库：`https://duckdb.org`
- **Joblib** 并行计算：`https://joblib.readthedocs.io`

谢谢！欢迎提问

ShenzhenLime